Подготовка к сдаче: WeakPtr и ЛР2

Сначала WeakPtr — это цель визита к магистрам. ЛР2 — консультация: «вот направление, оцените».

Общая архитектура

ДЗ-1, умные указатели

Три класса: `UniquePtr` (единственный владелец), `SharedPtr` (совместное владение, счётчик ссылок) и `WeakPtr` (наблюдает, но не владеет). У каждого `SharedPtr` есть указатель на **контрольный блок** — отдельный объект в куче, общий для всех владельцев одного объекта. Базовый класс блока `ControlBlockBase` нешаблонный: в нём два счётчика `strong` и `weak` и чисто виртуальный `DestroyObject()`. Шаблонный наследник `ControlBlock<U>` хранит внутри `UniquePtr<U>`, который и владеет объектом. Поэтому `delete`/`delete[]` во всём проекте ровно в одном месте — в `UniversalDelete` (функтор, специализация для `T[]`).

Главная мысль: **объект живёт, пока strong > 0; блок живёт, пока жив хоть кто-то — strong или weak**.

ЛР2, поиск

Две структуры для поиска за общими интерфейсами. `IDictionary` → `HashTable`: массив корзин, в корзине список пар, поиск в среднем O(1). `ISortedSequence` → `SortedSequence`: массив, всегда отсортированный, бинарный поиск O(log n) с настраиваемой пропорцией деления, умеет диапазон. Обе построены на моём `MutableArraySequence` с первого курса. `Person` — данные для будущего индекса. Индекса, замеров и меню пока нет.

WeakPtr — главные функции

Код ниже — схема по смыслу. Перед выходом сверь со своим `SharedPtr.h`.

1. ControlBlockBase и DestroyObject

```
struct ControlBlockBase {
    int strong = 1;          // сколько SharedPtr
    int weak = 0;            // сколько WeakPtr
    virtual void DestroyObject() = 0;
    virtual ~ControlBlockBase() = default;
};

template <class U>
struct ControlBlock : ControlBlockBase {
    UniquePtr<U> owner;
    void DestroyObject() override { owner.Reset(); }
};
```

Зачем. Два счётчика нужны, потому что у объекта и у блока **разное время жизни**. `DestroyObject` удаляет объект досрочно, а блок остаётся, чтобы `WeakPtr` мог прочитать «объект умер». Виртуальный — потому что базовый блок не знает тип объекта, а удалить правильно (`delete` или `delete[]`) может только шаблонный наследник.

Могут спросить:

— **Зачем два счётчика, а не один?** strong решает судьбу объекта, weak — судьбу блока. С одним счётчиком слабая ссылка либо держала бы объект живым, либо читала бы удалённую память.

— **Почему блок нешаблонный в основе?** Чтобы `SharedPtr<Derived>` и `SharedPtr<Base>` могли делить один блок (подтипизация).

2. SharedPtr::Release()

```
void Release() {
    if (!ctrl) return;
    ctrl->strong--;
    if (ctrl->strong == 0) {
        ctrl->DestroyObject();        // объект умер
        if (ctrl->weak == 0) {
            delete ctrl;              // и наблюдателей нет — удаляем блок
        }
    }
    ctrl = nullptr; ptr = nullptr;
}
```

Зачем. Вызывается из деструктора, `Reset` и `operator=`. Два **вложенных** условия: сначала объект, и только если он умер — смотрим на слабые ссылки.

Могут спросить:

— **Почему проверка weak внутри, а не рядом?** Пока strong > 0, блок нельзя удалять ни при каком weak.

— **Кто удалит блок, если weak > 0?** Последний WeakPtr в своём Release.

3. WeakPtr: конструктор от SharedPtr, Release

```
WeakPtr(const SharedPtr<T>& s) : ptr(s.ptr), ctrl(s.ctrl) {
    if (ctrl) ctrl->weak++;           // strong НЕ трогаем
}

void Release() {
    if (!ctrl) return;
    ctrl->weak--;
    if (ctrl->weak == 0 && ctrl->strong == 0) {
        delete ctrl;                  // объект трогать нельзя — он уже удалён
    }
    ctrl = nullptr; ptr = nullptr;
}
```

Зачем. WeakPtr **никогда** не удаляет объект и не продлевает ему жизнь — только weak. Блок удаляет последний, кто видит оба счётчика нулевыми.

Могут спросить:

— **Почему WeakPtr не увеличивает strong?** Тогда он владел бы объектом и цикл не разорвался бы.

— **Копирование и operator= у WeakPtr?** Копия: weak++. Присваивание: проверка на себя, Release старого, взять новый ptr/ctrl, weak++, return *this.

4. Expired и Lock

```
bool Expired() const { return !ctrl || ctrl->strong == 0; }

SharedPtr<T> Lock() const {
    return SharedPtr<T>(*this);       // конструктор SharedPtr из WeakPtr
}

// внутри SharedPtr:
SharedPtr(const WeakPtr<T>& w) : ptr(nullptr), ctrl(nullptr) {
    if (w.ctrl && w.ctrl->strong > 0) {   // ВАЖНО: w.ctrl, а не ctrl
        ptr = w.ptr; ctrl = w.ctrl;
        ctrl->strong++;
    }
}
```

Зачем. `Lock` — единственный безопасный способ добраться до объекта: сначала взять сильную ссылку, потом пользоваться. Пока результат `Lock` жив, объект не умрёт.

Могут спросить:

— **Почему нельзя просто разыменовать WeakPtr?** Между проверкой «жив?» и обращением объект мог умереть. Lock делает проверку и захват за один шаг.

— **Lock на просроченной?** Пустой SharedPtr. В стандарте конструктор из weak_ptr бросает bad_weak_ptr — у меня проще.

— **Расскажи про баг.** Было `if (ctrl && ctrl->strong > 0)` — проверялся свой ctrl, только что обнулённый. Lock всегда давал пустоту. Поймал: Lock().Get() печатал 0 на живом объекте.

5. Цикл и зачем всё это

`EmployeeCycle` с полем `SharedPtr colleague`: два объекта ссылаются друг на друга, у каждого strong ≥ 1 от соседа — после выхода из области видимости никто не удаляется, утечка. `Employee` с `WeakPtr colleague`: ссылка на коллегу не держит его, счётчики доходят до нуля, оба удаляются. В тесте утечку доказываю так: снаружи держу только один объект — второй обязан был умереть, а он жив.

Могут спросить:

— **Где ещё нужны слабые ссылки?** Наблюдатели (подписчики), кеши, ссылка ребёнка на родителя в дереве.

— **Почему нет WeakPtr<T[]>?** Удаление внутри блока, WeakPtr объект не трогает; у массивов нет связей между объектами, ради которых нужны слабые ссылки.

6. std::move — выучить дословно

«`std::move` — это приведение типа к rvalue-ссылке, `static_cast<T&&>`. В машинном коде ноль инструкций, сам ничего не перемещает. Он разрешает компилятору выбрать move-конструктор или move-присваивание, а уже они забирают указатель и обнуляют его у источника.» Не начинать со слова «перемещение».

ЛР2 — главные функции

HashTable::IndexFor и FindInBucket

```
int IndexFor(const TKey& key, int capacity) const {
    int h = hashFunc(key) % capacity;
    return h < 0 ? h + capacity : h;      // -7 % 10 == -7 в C++
}

int FindInBucket(const Bucket* bucket, const TKey& key) const {
    for (int i = 0; i < bucket->GetLength(); i++)
        if ((*bucket)[i].key == key) return i;   // по ссылке, без копии
    return -1;
}
```

Зачем. Номер корзины и общий поиск, на котором стоят `Get`, `ContainsKey`, `Remove`. Поправка на минус — потому что таблица не должна доверять чужой хеш-функции.

Могут спросить:

— **Сложность?** В среднем O(1), в худшем O(n) — все ключи в одной корзине.

— **Коллизия?** Два ключа в одной корзине; неизбежна, решена списками в корзинах.

HashTable::Add и Rehash

```
void Add(const TKey& key, const TElement& elem) {
    if (ContainsKey(key)) throw std::invalid_argument("KeyAlreadyExists");
    (*buckets)[IndexFor(key)]->Append(Pair<TKey, TElement>{key, elem});
    count++;
    if (count == buckets->GetLength()) Rehash(buckets->GetLength() * Q);
}

void Rehash(int newCapacity) {
    // новый массив пустых корзин, каждую пару — по IndexFor(key, newCapacity),
    // старые корзины и массив удалить, buckets = newBuckets
}
```

Зачем. Перестройка держит корзины короткими. Номер корзины зависит от числа корзин, поэтому **каждую** пару считаем заново от новой ёмкости. Сжатие в Remove при count ≤ capacity/4, но не ниже 4 корзин.

Могут спросить:

— **Почему p ≥ q (4 и 2)?** Иначе пинг-понг: сжались — сразу полные — снова расширение.

— **Почему buckets — указатель?** Чтобы в Rehash заменить массив целиком: удалить старый, переставить указатель. Присваивать последовательности было опасно — не было operator=.

— **Почему корзины на сырых указателях, не SharedPtr?** Владелец один — таблица; SharedPtr по смыслу не подходит и даёт оверхед счётчика. Правильнее UniquePtr; владение закрыто в классе, копирование запрещено, проверено ASan.

HashTable::Remove — swap-and-pop

```
int pos = FindInBucket(bucket, key);
if (pos == -1) throw std::out_of_range("KeyNotFound");
int last = bucket->GetLength() - 1;
if (pos != last) (*bucket)[pos] = std::move((*bucket)[last]);
bucket->RemoveLast();
```

Зачем. Последняя пара встаёт на место удаляемой, хвост отрезается. Без new/delete и без сдвигов. Можно, потому что порядок в корзине не важен.

SortedSequence::Bound — сердце бинарного поиска

```
int Bound(const T& x, bool upper) const {
    int lo = 0, hi = items.GetLength();
    while (lo < hi) {
        int mid = lo + (int)((long long)(hi - lo) * splitLeft / (splitLeft + splitRight));
        const T& m = items[mid];
```

```
        bool goRight = upper ? !(x < m) : (m < x);
        if (goRight) lo = mid + 1;   // mid точно не ответ — выкидываем с ним
        else         hi = mid;       // mid может быть ответом — оставляем
    }
    return lo;
}
```

Зачем. Один цикл для LowerBound (первый ≥ x) и UpperBound (первый > x) — разница только на равных x. Пропорция деления (1:1, 618:382, своя) меняет число шагов, но не ответ.

Могут спросить:

— **Сложность?** O(log n).

— **Какое деление быстрее?** Пополам. Золотое сечение в худшем случае ≈ 1.44·$\log_2$ n шагов.

— **Зачем long long?** При 10^8 элементов × 618 переполнится int.

— **Почему только оператор <?** ≤ это !(b<a), равенство !(a<b)&&!(b<a) — от типа нужен один оператор.

SortedSequence::Add, IndexOf, GetRange

```
void Add(const T& x) {
    int pos = UpperBound(x);                 // после равных
    items.Append(x);                         // удлинили
    for (int i = GetLength() - 1; i > pos; i--)
        items[i] = std::move(items[i - 1]);  // сдвиг справа налево
    items[pos] = x;
}
IndexOf:  i = LowerBound(x); i < длины и элемент равен x → i, иначе -1
GetRange(a, b):  от LowerBound(a) до UpperBound(b) - 1; пусто, если lo > hi
```

Могут спросить:

— **Сложность Add?** Поиск O(log n), но сдвиг O(n).

— **Почему items — поле, а не родитель?** Иначе снаружи были бы Append/InsertAt и можно сломать порядок. Инвариант «всегда отсортировано» держат конструктор (NotSorted) и Add.

— **GetRange возвращает указатель — почему?** Из интерфейса нельзя вернуть абстрактный тип по значению. Тип возврата ковариантный: SortedSequence<T>* вместо ISortedSequence<T>*.

Старый код: operator= и Append(T&&)

`DynamicArray::operator=` — правило трёх. Порядок: проверка `this == &other` → выделить и скопировать → `delete[]` старого → переставить поля → `return *this`. Нашёл санитайзером: без него `r = s.GetRange(...)` давал heap-use-after-free.

`Append(T&& item)` — внутри функции `item` уже lvalue (у него есть имя), поэтому нужен `std::move(item)`, иначе молча копирует. Нашли счётчиком копий.

Вопросы преподавателю (ЛР2)

1. Складываются ли клетка таблицы и строка метода — выходит 59 или около 51? 2. Можно ли теперь std::unique_ptr? 3. Можно ли добавить в IDictionary operator[] со ссылкой вместо копии в Get? 4. Индекс с повторяющимися ключами: «год → список номеров» — так нормально?